

Invariantes de Tutte-Grothendieck en grafos

María Alejandra Schild

Seminario de complejidad computacional

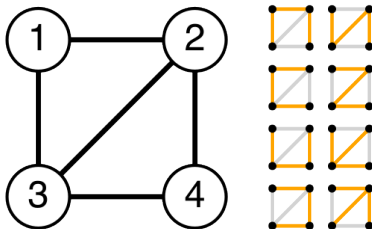
Septiembre de 2025

Índice

- 1 Problemas de conteo y la clase $\#P$
- 2 Las operaciones de supresión y contracción
- 3 El polinomio de Tutte

Un primer ejemplo: árboles generadores

Sea $G = (V, E)$ un grafo simple (sin loops ni aristas paralelas), no dirigido y conexo. Dado $T \subseteq E$, se dice que el subgrafo (V, T) es un **árbol generador** de G si es un árbol que conecta a todos los vértices de V .



User: Andreschulz, Wikimedia Commons, CC BY-SA 3.0

Todo árbol generador (V, T) cumple que $|T| = |V| - 1$.

Enumeración versus conteo

Problema de enumeración: «¿Cuáles?»

¿Existe un algoritmo de tiempo polinomial para **listar** los árboles generadores de G ?

Enumeración versus conteo

Problema de enumeración: «¿Cuáles?»

¿Existe un algoritmo de tiempo polinomial para **listar** los árboles generadores de G ?

No, porque puede haber demasiados.

Fórmula de Cayley

El grafo completo K_n (con vértices etiquetados) tiene exactamente n^{n-2} árboles generadores.

Enumeración versus conteo

Problema de enumeración: «¿Cuáles?»

¿Existe un algoritmo de tiempo polinomial para **listar** los árboles generadores de G ?

No, porque puede haber demasiados.

Fórmula de Cayley

El grafo completo K_n (con vértices etiquetados) tiene exactamente n^{n-2} árboles generadores.

Problema de conteo: «¿Cuántos?»

¿Existe un algoritmo de tiempo polinomial para **contar** los árboles generadores de G ?

Enumeración versus conteo

Problema de enumeración: «¿Cuáles?»

¿Existe un algoritmo de tiempo polinomial para **listar** los árboles generadores de G ?

No, porque puede haber demasiados.

Fórmula de Cayley

El grafo completo K_n (con vértices etiquetados) tiene exactamente n^{n-2} árboles generadores.

Problema de conteo: «¿Cuántos?»

¿Existe un algoritmo de tiempo polinomial para **contar** los árboles generadores de G ?

¡Sí!

El teorema de Kirchhoff

Digamos que $V = \{v_1, \dots, v_n\}$. La **matriz laplaciana** de $G = (V, E)$ es una matriz de $n \times n$ definida de la siguiente manera:

$$L_{i,j} := \begin{cases} \deg(v_i) & \text{si } i = j \\ -1 & \text{si } \{v_i, v_j\} \in E \\ 0 & \text{en otro caso.} \end{cases}$$

El teorema de Kirchhoff

Digamos que $V = \{v_1, \dots, v_n\}$. La **matriz laplaciana** de $G = (V, E)$ es una matriz de $n \times n$ definida de la siguiente manera:

$$L_{i,j} := \begin{cases} \deg(v_i) & \text{si } i = j \\ -1 & \text{si } \{v_i, v_j\} \in E \\ 0 & \text{en otro caso.} \end{cases}$$

Teorema de Kirchhoff

Sea L^* la matriz obtenida al borrar la primera fila y la primera columna de L . Entonces el número de árboles generadores de G es $\det(L^*)$.

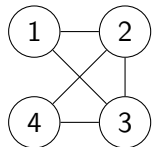
El teorema de Kirchhoff

Digamos que $V = \{v_1, \dots, v_n\}$. La **matriz laplaciana** de $G = (V, E)$ es una matriz de $n \times n$ definida de la siguiente manera:

$$L_{i,j} := \begin{cases} \deg(v_i) & \text{si } i = j \\ -1 & \text{si } \{v_i, v_j\} \in E \\ 0 & \text{en otro caso.} \end{cases}$$

Teorema de Kirchhoff

Sea L^* la matriz obtenida al borrar la primera fila y la primera columna de L . Entonces el número de árboles generadores de G es $\det(L^*)$.



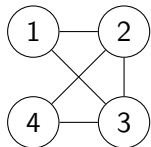
El teorema de Kirchhoff

Digamos que $V = \{v_1, \dots, v_n\}$. La **matriz laplaciana** de $G = (V, E)$ es una matriz de $n \times n$ definida de la siguiente manera:

$$L_{ij} := \begin{cases} \deg(v_i) & \text{si } i = j \\ -1 & \text{si } \{v_i, v_j\} \in E \\ 0 & \text{en otro caso.} \end{cases}$$

Teorema de Kirchhoff

Sea L^* la matriz obtenida al borrar la primera fila y la primera columna de L . Entonces el número de árboles generadores de G es $\det(L^*)$.



$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 2 \end{pmatrix}$$

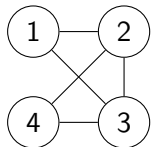
El teorema de Kirchhoff

Digamos que $V = \{v_1, \dots, v_n\}$. La **matriz laplaciana** de $G = (V, E)$ es una matriz de $n \times n$ definida de la siguiente manera:

$$L_{ij} := \begin{cases} \deg(v_i) & \text{si } i = j \\ -1 & \text{si } \{v_i, v_j\} \in E \\ 0 & \text{en otro caso.} \end{cases}$$

Teorema de Kirchhoff

Sea L^* la matriz obtenida al borrar la primera fila y la primera columna de L . Entonces el número de árboles generadores de G es $\det(L^*)$.



$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 2 \end{pmatrix} \quad \left| \begin{array}{ccc} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 2 \end{array} \right| = 8$$

Un segundo ejemplo: coloraciones de vértices

Sea $G = (V, E)$ un grafo simple y no dirigido, y sea $k \geq 1$ un entero. Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para todos $u, v \in V$ tales que $\{u, v\} \in E$.

Un segundo ejemplo: coloraciones de vértices

Sea $G = (V, E)$ un grafo simple y no dirigido, y sea $k \geq 1$ un entero. Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para todos $u, v \in V$ tales que $\{u, v\} \in E$.

¿Hay un algoritmo de tiempo polinomial para contar 1-coloraciones de G ?

Un segundo ejemplo: coloraciones de vértices

Sea $G = (V, E)$ un grafo simple y no dirigido, y sea $k \geq 1$ un entero. Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para todos $u, v \in V$ tales que $\{u, v\} \in E$.

¿Hay un algoritmo de tiempo polinomial para contar 1-coloraciones de G ?

Sí. Son 0 si $E \neq \emptyset$, y 1 si $E = \emptyset$.

Un segundo ejemplo: coloraciones de vértices

Sea $G = (V, E)$ un grafo simple y no dirigido, y sea $k \geq 1$ un entero. Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para todos $u, v \in V$ tales que $\{u, v\} \in E$.

¿Hay un algoritmo de tiempo polinomial para contar 1-coloraciones de G ?

Sí. Son 0 si $E \neq \emptyset$, y 1 si $E = \emptyset$.

¿Hay un algoritmo de tiempo polinomial para contar 2-coloraciones de G ?

Un segundo ejemplo: coloraciones de vértices

Sea $G = (V, E)$ un grafo simple y no dirigido, y sea $k \geq 1$ un entero. Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para todos $u, v \in V$ tales que $\{u, v\} \in E$.

¿Hay un algoritmo de tiempo polinomial para contar 1-coloraciones de G ?

Sí. Son 0 si $E \neq \emptyset$, y 1 si $E = \emptyset$.

¿Hay un algoritmo de tiempo polinomial para contar 2-coloraciones de G ?

Sí. Si G no es bipartito, no tiene 2-coloraciones. Si G es bipartito con $\kappa(G)$ componentes conexas, entonces el número de 2-coloraciones de G es $2^{\kappa(G)}$.

Un segundo ejemplo: coloraciones de vértices

Sea $G = (V, E)$ un grafo simple y no dirigido, y sea $k \geq 1$ un entero. Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para todos $u, v \in V$ tales que $\{u, v\} \in E$.

¿Hay un algoritmo de tiempo polinomial para contar 1-coloraciones de G ?

Sí. Son 0 si $E \neq \emptyset$, y 1 si $E = \emptyset$.

¿Hay un algoritmo de tiempo polinomial para contar 2-coloraciones de G ?

Sí. Si G no es bipartito, no tiene 2-coloraciones. Si G es bipartito con $\kappa(G)$ componentes conexas, entonces el número de 2-coloraciones de G es $2^{\kappa(G)}$.

¿Hay un algoritmo de tiempo polinomial para contar 3-coloraciones de G ?

Un segundo ejemplo: coloraciones de vértices

Sea $G = (V, E)$ un grafo simple y no dirigido, y sea $k \geq 1$ un entero. Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que $c(u) \neq c(v)$ para todos $u, v \in V$ tales que $\{u, v\} \in E$.

¿Hay un algoritmo de tiempo polinomial para contar 1-coloraciones de G ?

Sí. Son 0 si $E \neq \emptyset$, y 1 si $E = \emptyset$.

¿Hay un algoritmo de tiempo polinomial para contar 2-coloraciones de G ?

Sí. Si G no es bipartito, no tiene 2-coloraciones. Si G es bipartito con $\kappa(G)$ componentes conexas, entonces el número de 2-coloraciones de G es $2^{\kappa(G)}$.

¿Hay un algoritmo de tiempo polinomial para contar 3-coloraciones de G ?

Si asumimos que $P \neq NP$, entonces **no**, pues incluso **decidir** si existe una 3-coloración es NP-completo.

Problemas de conteo de tiempo polinomial

Un **problema de conteo** es una función $f: \{0, 1\}^* \rightarrow \mathbb{N}$.

Problemas de conteo de tiempo polinomial

Un **problema de conteo** es una función $f: \{0,1\}^* \rightarrow \mathbb{N}$.

Diremos que un problema de conteo $f: \{0,1\}^* \rightarrow \mathbb{N}$ está en la clase **FP** si existe una MTC que funciona en tiempo polinomial y que, con entrada $w \in \{0,1\}^*$, calcula la representación binaria de $f(w)$.

Problemas de conteo de tiempo polinomial

Un **problema de conteo** es una función $f: \{0,1\}^* \rightarrow \mathbb{N}$.

Diremos que un problema de conteo $f: \{0,1\}^* \rightarrow \mathbb{N}$ está en la clase **FP** si existe una MTC que funciona en tiempo polinomial y que, con entrada $w \in \{0,1\}^*$, calcula la representación binaria de $f(w)$.

Ejemplos:

- Contar árboles generadores está en FP.
- Si $P \neq NP$, entonces contar 3-coloraciones no está en FP.

Ojo con la definición de FP

La clase FP realmente consiste de todas las funciones $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ computables en tiempo polinomial. La salida podría naturalmente codificar otro tipo de objetos.

Ejemplos:

- Sumar dos números racionales.
- Multiplicar dos matrices con entradas enteras.

Ojo con la definición de FP

La clase FP realmente consiste de todas las funciones $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ computables en tiempo polinomial. La salida podría naturalmente codificar otro tipo de objetos.

Ejemplos:

- Sumar dos números racionales.
- Multiplicar dos matrices con entradas enteras.

Pero podemos transformar cualquier $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ en un problema de conteo $f': \{0, 1\}^* \rightarrow \mathbb{N}$ de modo que, si $f(w) = a_k a_{k-1} \dots a_1 \in \{0, 1\}^k$, entonces

$$f'(w) := 2^k + \sum_{i=1}^k 2^{i-1} a_i.$$

Notemos que $f \in \text{FP}$ si y solo si $f' \in \text{FP}$.

La clase #P

Al decir que una MTND M funciona en tiempo polinomial suponemos que existe un polinomio $p(\cdot)$ tal que, para cada $w \in \{0, 1\}^*$, el número de pasos de **cualquier** ejecución de M con entrada w es a lo más $p(|w|)$.

La clase #P

Al decir que una MTND M funciona en tiempo polinomial suponemos que existe un polinomio $p(\cdot)$ tal que, para cada $w \in \{0, 1\}^*$, el número de pasos de **cualquier** ejecución de M con entrada w es a lo más $p(|w|)$.

Dado $w \in \{0, 1\}^*$, definimos $\text{acc}_M(w)$ como el número de ejecuciones de aceptación de M con entrada w .

La clase #P

Al decir que una MTND M funciona en tiempo polinomial suponemos que existe un polinomio $p(\cdot)$ tal que, para cada $w \in \{0, 1\}^*$, el número de pasos de **cualquier** ejecución de M con entrada w es a lo más $p(|w|)$.

Dado $w \in \{0, 1\}^*$, definimos $\text{acc}_M(w)$ como el número de ejecuciones de aceptación de M con entrada w .

Definición de la clase #P

Una función $f : \{0, 1\}^* \rightarrow \mathbb{N}$ está en #P si y sólo si existe una MTND M que funciona en tiempo polinomial y tal que, para cada $w \in \{0, 1\}^*$, se cumple que $f(w) = \text{acc}_M(w)$.

La clase #P

Al decir que una MTND M funciona en tiempo polinomial suponemos que existe un polinomio $p(\cdot)$ tal que, para cada $w \in \{0, 1\}^*$, el número de pasos de **cualquier** ejecución de M con entrada w es a lo más $p(|w|)$.

Dado $w \in \{0, 1\}^*$, definimos $\text{acc}_M(w)$ como el número de ejecuciones de aceptación de M con entrada w .

Definición de la clase #P

Una función $f : \{0, 1\}^* \rightarrow \mathbb{N}$ está en #P si y sólo si existe una MTND M que funciona en tiempo polinomial y tal que, para cada $w \in \{0, 1\}^*$, se cumple que $f(w) = \text{acc}_M(w)$.

#3SAT, el problema de contar las valuaciones que satisfacen una fórmula proposicional en 3CNF, está en #P.

$\#P$ es cerrada bajo sumas y multiplicaciones

Sean $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ funciones en $\#P$. Entonces:

1. $f + g \in \#P$.
2. $f \cdot g \in \#P$.

$\#P$ es cerrada bajo sumas y multiplicaciones

Sean $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ funciones en $\#P$. Entonces:

1. $f + g \in \#P$.
2. $f \cdot g \in \#P$.

Demostración.

$f \in \#P$ via una MTND M_1 y $g \in \#P$ via otra MTND M_2 .

$\#P$ es cerrada bajo sumas y multiplicaciones

Sean $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ funciones en $\#P$. Entonces:

1. $f + g \in \#P$.
2. $f \cdot g \in \#P$.

Demostración.

$f \in \#P$ via una MTND M_1 y $g \in \#P$ via otra MTND M_2 .

- Para (1), consideramos una MTND que, con entrada $w \in \{0, 1\}^*$, adivina un bit $b \in \{0, 1\}$, simula $M_{b+1}(w)$ y retorna la respuesta de dicha simulación.

$\#P$ es cerrada bajo sumas y multiplicaciones

Sean $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ funciones en $\#P$. Entonces:

1. $f + g \in \#P$.
2. $f \cdot g \in \#P$.

Demostración.

$f \in \#P$ via una MTND M_1 y $g \in \#P$ via otra MTND M_2 .

- Para (1), consideramos una MTND que, con entrada $w \in \{0, 1\}^*$, adivina un bit $b \in \{0, 1\}$, simula $M_{b+1}(w)$ y retorna la respuesta de dicha simulación.
- Para (2), consideramos una MTND M_3 que, con entrada $w \in \{0, 1\}^*$, simula $M_1(w)$. Si la simulación rechaza, entonces M_3 rechaza; si la simulación acepta, entonces M_3 simula $M_2(w)$ y retorna la respuesta de dicha simulación.



FP versus #P

Para comparar las clases FP y #P, identificamos cada elemento de $\mathbb{N}$ con su representación binaria. Recíprocamente, identificamos cada elemento de $\{0, 1\}^*$ con un número natural, ignorando los ceros a la izquierda.

FP versus #P

Para comparar las clases FP y #P, identificamos cada elemento de $\mathbb{N}$ con su representación binaria. Recíprocamente, identificamos cada elemento de $\{0,1\}^*$ con un número natural, ignorando los ceros a la izquierda.

FP $\subseteq$ #P, y la inclusión es estricta si $P \neq NP$.

FP versus #P

Para comparar las clases FP y #P, identificamos cada elemento de $\mathbb{N}$ con su representación binaria. Recíprocamente, identificamos cada elemento de $\{0,1\}^*$ con un número natural, ignorando los ceros a la izquierda.

$FP \subseteq \#P$, y la inclusión es estricta si $P \neq NP$.

Demostración.

Sea $f \in FP$. Notemos que $|f(w)| = \text{poly}(|w|)$.

FP versus #P

Para comparar las clases FP y #P, identificamos cada elemento de $\mathbb{N}$ con su representación binaria. Recíprocamente, identificamos cada elemento de $\{0, 1\}^*$ con un número natural, ignorando los ceros a la izquierda.

$FP \subseteq \#P$, y la inclusión es estricta si $P \neq NP$.

Demostración.

Sea $f \in FP$. Notemos que $|f(w)| = \text{poly}(|w|)$. Sea M una MTND que, con entrada $w \in \{0, 1\}^*$, calcula $f(w)$, adivina una palabra $z \in \{0, 1\}^{|f(w)|}$ y acepta si y solo si z es menor que $f(w)$ en orden lexicográfico (considerando el bit más significativo a la izquierda).

FP versus #P

Para comparar las clases FP y #P, identificamos cada elemento de $\mathbb{N}$ con su representación binaria. Recíprocamente, identificamos cada elemento de $\{0, 1\}^*$ con un número natural, ignorando los ceros a la izquierda.

$FP \subseteq \#P$, y la inclusión es estricta si $P \neq NP$.

Demostración.

Sea $f \in FP$. Notemos que $|f(w)| = \text{poly}(|w|)$. Sea M una MTND que, con entrada $w \in \{0, 1\}^*$, calcula $f(w)$, adivina una palabra $z \in \{0, 1\}^{|f(w)|}$ y acepta si y solo si z es menor que $f(w)$ en orden lexicográfico (considerando el bit más significativo a la izquierda). Entonces $f \in \#P$ via M .

FP versus #P

Para comparar las clases FP y #P, identificamos cada elemento de $\mathbb{N}$ con su representación binaria. Recíprocamente, identificamos cada elemento de $\{0, 1\}^*$ con un número natural, ignorando los ceros a la izquierda.

FP $\subseteq$ #P, y la inclusión es estricta si $P \neq NP$.

Demostración.

Sea $f \in \text{FP}$. Notemos que $|f(w)| = \text{poly}(|w|)$. Sea M una MTND que, con entrada $w \in \{0, 1\}^*$, calcula $f(w)$, adivina una palabra $z \in \{0, 1\}^{|f(w)|}$ y acepta si y solo si z es menor que $f(w)$ en orden lexicográfico (considerando el bit más significativo a la izquierda). Entonces $f \in \text{\#P}$ via M .

Para la segunda parte, si $\text{\#P} = \text{FP}$, entonces $\text{\#3SAT} \in \text{FP}$, y tendríamos un algoritmo de tiempo polinomial para decidir 3SAT. □

FP versus #P

Para comparar las clases FP y #P, identificamos cada elemento de $\mathbb{N}$ con su representación binaria. Recíprocamente, identificamos cada elemento de $\{0, 1\}^*$ con un número natural, ignorando los ceros a la izquierda.

FP $\subseteq$ #P, y la inclusión es estricta si $P \neq NP$.

Demostración.

Sea $f \in \text{FP}$. Notemos que $|f(w)| = \text{poly}(|w|)$. Sea M una MTND que, con entrada $w \in \{0, 1\}^*$, calcula $f(w)$, adivina una palabra $z \in \{0, 1\}^{|f(w)|}$ y acepta si y solo si z es menor que $f(w)$ en orden lexicográfico (considerando el bit más significativo a la izquierda). Entonces $f \in \text{\#P}$ via M .

Para la segunda parte, si $\text{\#P} = \text{FP}$, entonces $\text{\#3SAT} \in \text{FP}$, y tendríamos un algoritmo de tiempo polinomial para decidir 3SAT. □

La igualdad $\text{\#P} = \text{FP}$ se considera más catastrófica que $NP = P$.

Una primera noción de reducción para funciones

Reducciones parsimoniosas

Dadas funciones $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, decimos que f se puede reducir parsimoniosamente a g (en tiempo polinomial), denotado como $f \leq_{par}^p g$, si existe $h \in \text{FP}$ tal que $f(w) = g(h(w))$ para todo $w \in \{0, 1\}^*$.

Una primera noción de reducción para funciones

Reducciones parsimoniosas

Dadas funciones $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, decimos que f se puede reducir parsimoniosamente a g (en tiempo polinomial), denotado como $f \leq_{par}^P g$, si existe $h \in \text{FP}$ tal que $f(w) = g(h(w))$ para todo $w \in \{0, 1\}^*$.

$\#3\text{SAT}$ es $\#P$ -completo bajo reducciones parsimoniosas.

Una primera noción de reducción para funciones

Reducciones parsimoniosas

Dadas funciones $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, decimos que f se puede reducir parsimoniosamente a g (en tiempo polinomial), denotado como $f \leq_{par}^P g$, si existe $h \in \text{FP}$ tal que $f(w) = g(h(w))$ para todo $w \in \{0, 1\}^*$.

$\#3\text{SAT}$ es $\#P$ -completo bajo reducciones parsimoniosas.

La función $\#3\text{SAT} + 1$ está en $\#P$ por ser suma de dos funciones en $\#P$, pero **no** es $\#P$ -completa bajo reducciones parsimoniosas.

Una primera noción de reducción para funciones

Reducciones parsimoniosas

Dadas funciones $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, decimos que f se puede reducir parsimoniosamente a g (en tiempo polinomial), denotado como $f \leq_{par}^P g$, si existe $h \in \text{FP}$ tal que $f(w) = g(h(w))$ para todo $w \in \{0, 1\}^*$.

$\#3\text{SAT}$ es $\#P$ -completo bajo reducciones parsimoniosas.

La función $\#3\text{SAT} + 1$ está en $\#P$ por ser suma de dos funciones en $\#P$, pero **no** es $\#P$ -completa bajo reducciones parsimoniosas.

¡Pero $\#3\text{SAT}$ y $\#3\text{SAT} + 1$ debiesen ser igual de difíciles de calcular!

Oráculos funcionales

Máquinas con oráculos

Sea $f: \{0,1\}^* \rightarrow \{0,1\}^*$ una función. Una MT con un **oráculo para f** cuenta con una cinta especial y un estado especial $q_?$. Al entrar al estado $q_?$, si $x \in \{0,1\}^*$ es el contenido actual de la cinta especial, entonces en un paso este cambia a $f(x) \in \{0,1\}^*$ y la máquina sale del estado $q_?$.

Oráculos funcionales

Máquinas con oráculos

Sea $f: \{0,1\}^* \rightarrow \{0,1\}^*$ una función. Una MT con un **oráculo para f** cuenta con una cinta especial y un estado especial $q_?$. Al entrar al estado $q_?$, si $x \in \{0,1\}^*$ es el contenido actual de la cinta especial, entonces en un paso este cambia a $f(x) \in \{0,1\}^*$ y la máquina sale del estado $q_?$.

Clases de complejidad con oráculos

FP^f es el conjunto de funciones de $\{0,1\}^*$ en $\{0,1\}^*$ computables en tiempo polinomial por una MTC con acceso a un oráculo para f .

Oráculos funcionales

Máquinas con oráculos

Sea $f: \{0,1\}^* \rightarrow \{0,1\}^*$ una función. Una MT con un **oráculo para f** cuenta con una cinta especial y un estado especial $q_?$. Al entrar al estado $q_?$, si $x \in \{0,1\}^*$ es el contenido actual de la cinta especial, entonces en un paso este cambia a $f(x) \in \{0,1\}^*$ y la máquina sale del estado $q_?$.

Clases de complejidad con oráculos

FP ^{f} es el conjunto de funciones de $\{0,1\}^*$ en $\{0,1\}^*$ computables en tiempo polinomial por una MTC con acceso a un oráculo para f .

Dada una clase de complejidad de funciones $\mathcal{C}$, definimos

$$\mathbf{FP}^{\mathcal{C}} := \bigcup_{f \in \mathcal{C}} \mathbf{FP}^f.$$

Oráculos funcionales

Máquinas con oráculos

Sea $f: \{0,1\}^* \rightarrow \{0,1\}^*$ una función. Una MT con un **oráculo para f** cuenta con una cinta especial y un estado especial $q_?$. Al entrar al estado $q_?$, si $x \in \{0,1\}^*$ es el contenido actual de la cinta especial, entonces en un paso este cambia a $f(x) \in \{0,1\}^*$ y la máquina sale del estado $q_?$.

Clases de complejidad con oráculos

FP^f es el conjunto de funciones de $\{0,1\}^*$ en $\{0,1\}^*$ computables en tiempo polinomial por una MTC con acceso a un oráculo para f .
Dada una clase de complejidad de funciones $\mathcal{C}$, definimos

$$\text{FP}^{\mathcal{C}} := \bigcup_{f \in \mathcal{C}} \text{FP}^f.$$

Ejercicio: Si $f \in \text{FP}$, entonces $\text{FP}^f = \text{FP}$.

Una noción de reducción más flexible para funciones

Reducciones de Turing

Dadas $f, g: \{0, 1\}^* \rightarrow \{0, 1\}^*$, decimos que f es Turing-reducible a g (en tiempo polinomial), denotado $f \leq_T^p g$, si $f \in \text{FP}^g$.

Una noción de reducción más flexible para funciones

Reducciones de Turing

Dadas $f, g: \{0, 1\}^* \rightarrow \{0, 1\}^*$, decimos que f es Turing-reducible a g (en tiempo polinomial), denotado $f \leq_T^p g$, si $f \in \text{FP}^g$.

Toda reducción parsimoniosa es una reducción de Turing, pero no al revés.

Una noción de reducción más flexible para funciones

Reducciones de Turing

Dadas $f, g: \{0, 1\}^* \rightarrow \{0, 1\}^*$, decimos que f es Turing-reducible a g (en tiempo polinomial), denotado $f \leq_T^P g$, si $f \in \text{FP}^g$.

Toda reducción parsimoniosa es una reducción de Turing, pero no al revés.

Ejercicio

Si $f_1 \leq_T^P f_2$ y $f_2 \leq_T^P f_3$, entonces $f_1 \leq_T^P f_3$.

Una mejor noción de #P-hardness

#P-hardness y #P-completitud

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es **#P-hard** (bajo reducciones de Turing) si $\#P \subseteq \text{FP}^f$.

Una mejor noción de #P-hardness

#P-hardness y #P-completitud

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es **#P-hard** (bajo reducciones de Turing) si $\#P \subseteq \text{FP}^f$.
- Un problema de conteo $f: \{0,1\}^* \rightarrow \mathbb{N}$ es **#P-completo** si $f \in \#P$ y es #P-hard.

Una mejor noción de #P-hardness

#P-hardness y #P-completitud

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es **#P-hard** (bajo reducciones de Turing) si $\#P \subseteq \text{FP}^f$.
- Un problema de conteo $f: \{0,1\}^* \rightarrow \mathbb{N}$ es **#P-completo** si $f \in \#P$ y es #P-hard.

Si $\text{FP} \neq \#P$, entonces **ninguna** función #P-hard puede ser calculada en tiempo polinomial.

Una mejor noción de #P-hardness

#P-hardness y #P-completitud

- Una función $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ es **#P-hard** (bajo reducciones de Turing) si $\#P \subseteq \text{FP}^f$.
- Un problema de conteo $f: \{0, 1\}^* \rightarrow \mathbb{N}$ es **#P-completo** si $f \in \#P$ y es #P-hard.

Si $\text{FP} \neq \#P$, entonces **ninguna** función #P-hard puede ser calculada en tiempo polinomial.

#DNF es #P-completo.

No puede ocurrir que $\#3\text{SAT} \leq_{\text{par}}^P \# \text{DNF}$ salvo que $P = \text{NP}$.

Una mejor noción de #P-hardness

#P-hardness y #P-completitud

- Una función $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ es **#P-hard** (bajo reducciones de Turing) si $\#P \subseteq FP^f$.
- Un problema de conteo $f: \{0, 1\}^* \rightarrow \mathbb{N}$ es **#P-completo** si $f \in \#P$ y es #P-hard.

Si $FP \neq \#P$, entonces **ninguna** función #P-hard puede ser calculada en tiempo polinomial.

#DNF es #P-completo.

No puede ocurrir que $\#3SAT \leq_{par}^P \#DNF$ salvo que $P = NP$.

Pero sí tenemos que $\#3SAT \leq_T^P \#DNF$: dada φ en 3CNF con n variables, podemos escribir una DNF equivalente a $\neg\varphi$ y luego aprovechar que el número de valuaciones que satisfacen φ es $2^n - \#DNF(\neg\varphi)$.

El poder de $\#P$

$\#P$ -easiness y $\#P$ -equivalencia

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es $\#P$ -easy si $f \in FP^{\#P}$.

El poder de $\#P$

$\#P$ -easiness y $\#P$ -equivalencia

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es $\#P$ -easy si $f \in FP^{\#P}$.
- Si f es a la vez $\#P$ -easy y $\#P$ -hard, se dice $\#P$ -equivalente.

El poder de $\#P$

$\#P$ -easiness y $\#P$ -equivalencia

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es **$\#P$ -easy** si $f \in FP^{\#P}$.
- Si f es a la vez $\#P$ -easy y $\#P$ -hard, se dice **$\#P$ -equivalente**.

Si $FP = \#P$, entonces **todas** las funciones $\#P$ -easy pueden ser calculadas en tiempo polinomial.

El poder de $\#P$

$\#P$ -easiness y $\#P$ -equivalencia

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es **$\#P$ -easy** si $f \in FP^{\#P}$.
- Si f es a la vez $\#P$ -easy y $\#P$ -hard, se dice **$\#P$ -equivalente**.

Si $FP = \#P$, entonces **todas** las funciones $\#P$ -easy pueden ser calculadas en tiempo polinomial.

Ejemplos: Los siguientes problemas de decisión son $\#P$ -easy:

- Dada una fórmula proposicional, decidir si el número de valuaciones que la satisfacen es impar (problema $\oplus P$ -completo).
- Dada una fórmula proposicional, decidir si más de la mitad de las valuaciones posibles la satisfacen (problema PP -completo).

El poder de $\#P$

$\#P$ -easiness y $\#P$ -equivalencia

- Una función $f: \{0,1\}^* \rightarrow \{0,1\}^*$ es **$\#P$ -easy** si $f \in FP^{\#P}$.
- Si f es a la vez $\#P$ -easy y $\#P$ -hard, se dice **$\#P$ -equivalente**.

Si $FP = \#P$, entonces **todas** las funciones $\#P$ -easy pueden ser calculadas en tiempo polinomial.

Ejemplos: Los siguientes problemas de decisión son $\#P$ -easy:

- Dada una fórmula proposicional, decidir si el número de valuaciones que la satisfacen es impar (problema $\oplus P$ -completo).
- Dada una fórmula proposicional, decidir si más de la mitad de las valuaciones posibles la satisfacen (problema PP -completo).

Teorema de Toda

Todos los problemas de decisión de la jerarquía polinomial son $\#P$ -easy.

La función cromática

Desde ahora, $G = (V, E)$ será siempre un grafo no dirigido (no necesariamente simple) con $|V| = n = n(G)$ y $|E| = m = m(G)$.

La función cromática

Desde ahora, $G = (V, E)$ será siempre un grafo no dirigido (no necesariamente simple) con $|V| = n = n(G)$ y $|E| = m = m(G)$.

Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que para toda arista con extremos $u, v \in V$ se cumple que $c(u) \neq c(v)$.

Al número de k -coloraciones de G lo denotaremos $\chi_G(k)$.

La función cromática

Desde ahora, $G = (V, E)$ será siempre un grafo no dirigido (no necesariamente simple) con $|V| = n = n(G)$ y $|E| = m = m(G)$.

Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que para toda arista con extremos $u, v \in V$ se cumple que $c(u) \neq c(v)$.

Al número de k -coloraciones de G lo denotaremos $\chi_G(k)$.

Evaluar $\chi_G(k)$ es FP si $k \in \{1, 2\}$, y es $\#P$ -completo si $k \geq 3$.

La función cromática

Desde ahora, $G = (V, E)$ será siempre un grafo no dirigido (no necesariamente simple) con $|V| = n = n(G)$ y $|E| = m = m(G)$.

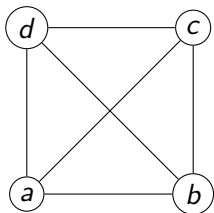
Una **k -coloración (de vértices)** de G es una función $c: V \rightarrow \{1, \dots, k\}$ tal que para toda arista con extremos $u, v \in V$ se cumple que $c(u) \neq c(v)$.

Al número de k -coloraciones de G lo denotaremos $\chi_G(k)$.

Evaluar $\chi_G(k)$ es FP si $k \in \{1, 2\}$, y es $\#P$ -completo si $k \geq 3$.

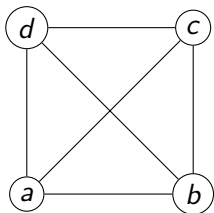
$\chi_G(k) \leq k^n$, con igualdad si y solo si $m = 0$.

Función cromática de un grafo completo



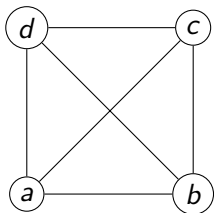
k	$\chi_G(k)$
-----	-------------

Función cromática de un grafo completo



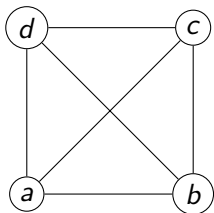
k	$\chi_G(k)$
1	0

Función cromática de un grafo completo



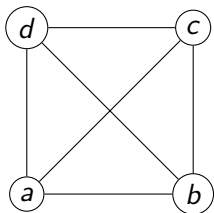
k	$\chi_G(k)$
1	0
2	0

Función cromática de un grafo completo



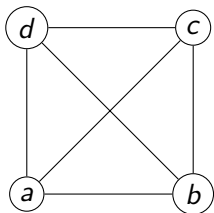
k	$\chi_G(k)$
1	0
2	0
3	0

Función cromática de un grafo completo



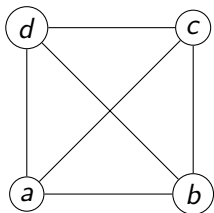
k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$

Función cromática de un grafo completo



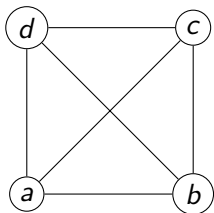
k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$
5	$5 \cdot 4 \cdot 3 \cdot 2$

Función cromática de un grafo completo



k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$
5	$5 \cdot 4 \cdot 3 \cdot 2$
6	$6 \cdot 5 \cdot 4 \cdot 3$

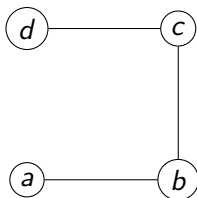
Función cromática de un grafo completo



k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$
5	$5 \cdot 4 \cdot 3 \cdot 2$
6	$6 \cdot 5 \cdot 4 \cdot 3$

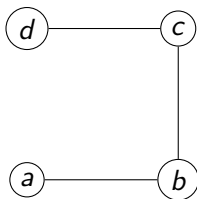
$$\chi_G(k) = k(k-1)(k-2)(k-3).$$

Más ejemplos de funciones cromáticas

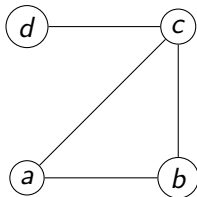


$$\chi_G(k) = k(k-1)^3$$

Más ejemplos de funciones cromáticas

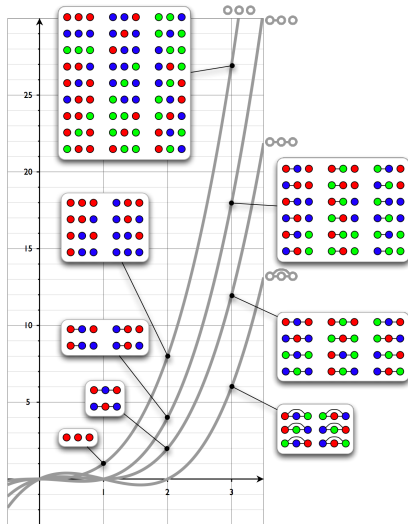


$$\chi_G(k) = k(k-1)^3$$



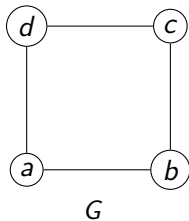
$$\chi_G(k) = k(k-1)^2(k-2)$$

¿Acaso las funciones cromáticas son siempre polinomios?

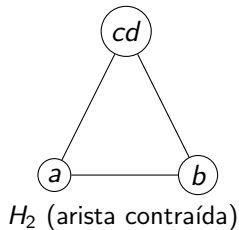
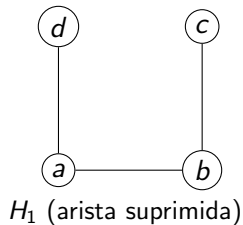
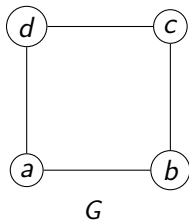


User: Thore Husfeldt, Wikimedia Commons, CC BY-SA 4.0

Un ejemplo más complejo

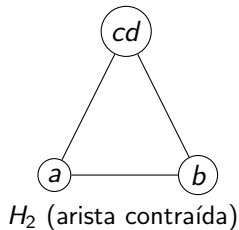
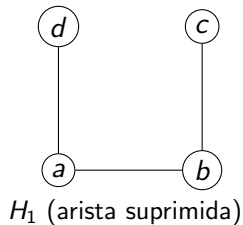
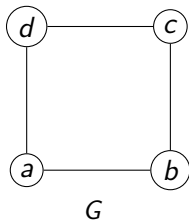


Un ejemplo más complejo



$$\chi_{H_1}(k) = \chi_G(k) + \chi_{H_2}(k)$$

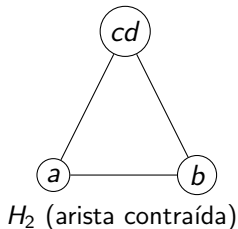
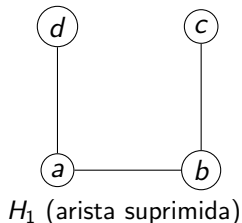
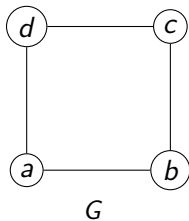
Un ejemplo más complejo



$$\chi_{H_1}(k) = \chi_G(k) + \chi_{H_2}(k)$$

$$\chi_G(k) = \chi_{H_1}(k) - \chi_{H_2}(k)$$

Un ejemplo más complejo



$$\chi_{H_1}(k) = \chi_G(k) + \chi_{H_2}(k)$$

$$\chi_G(k) = \chi_{H_1}(k) - \chi_{H_2}(k)$$

Como $m(H_1) = m(H_2) = m(G) - 1$, tenemos una **recursión**.

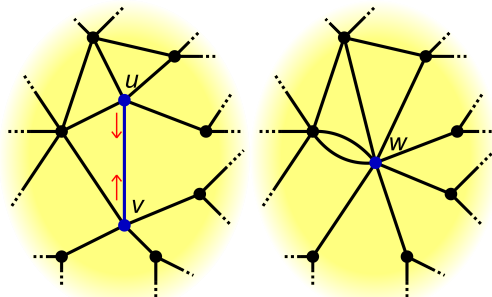
Las operaciones de supresión y contracción

Dada una arista $e \in E$, la **supresión** $G \setminus e$ es el grafo $(V, E \setminus e)$.

Las operaciones de supresión y contracción

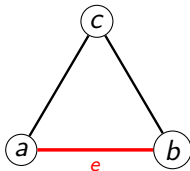
Dada una arista $e \in E$, la **supresión** $G \setminus e$ es el grafo $(V, E \setminus e)$.

Dada una arista $e \in E$ que conecta vértices distintos u y v , la **contracción** G/e es el grafo obtenido al borrar la arista e y luego identificar u y v como un solo nuevo vértice w .

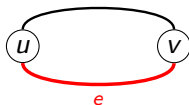


Cuidado con los loops

Contraer una arista de un triángulo crea **aristas paralelas**

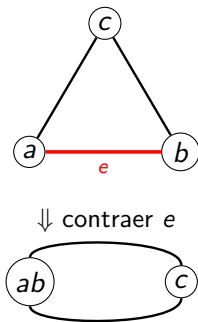


Contraer una arista que tenía otras aristas paralelas crea **loops**

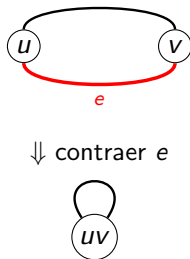


Cuidado con los loops

Contraer una arista de un triángulo crea **aristas paralelas**



Contraer una arista que tenía otras aristas paralelas crea **loops**



Si G es un grafo con un loop, entonces $\chi_G(k) = 0$.

¡La función cromática es un polinomio!

Recursión fundamental de la función cromática

$$\chi_G(k) = \begin{cases} k^{n(G)} & \text{si } m(G) = 0 \\ 0 & \text{si } G \text{ tiene loops} \\ \chi_{G \setminus e}(k) - \chi_{G/e}(k) & \text{si } e \text{ es (cualquier) arista no loop} \end{cases} .$$

¡La función cromática es un polinomio!

Recursión fundamental de la función cromática

$$\chi_G(k) = \begin{cases} k^{n(G)} & \text{si } m(G) = 0 \\ 0 & \text{si } G \text{ tiene loops} \\ \chi_{G \setminus e}(k) - \chi_{G/e}(k) & \text{si } e \text{ es (cualquier) arista no loop} \end{cases} .$$

Corolario

Dado cualquier grafo G sin loops, la función $\chi_G(k)$ coincide con un polinomio en $\mathbb{Z}[k]$ sin término constante y cuyo término líder es $k^{n(G)}$.

¡La función cromática es un polinomio!

Recursión fundamental de la función cromática

$$\chi_G(k) = \begin{cases} k^{n(G)} & \text{si } m(G) = 0 \\ 0 & \text{si } G \text{ tiene loops} \\ \chi_{G \setminus e}(k) - \chi_{G/e}(k) & \text{si } e \text{ es (cualquier) arista no loop} \end{cases} .$$

Corolario

Dado cualquier grafo G sin loops, la función $\chi_G(k)$ coincide con un polinomio en $\mathbb{Z}[k]$ sin término constante y cuyo término líder es $k^{n(G)}$.

Realmente a $\chi_G(k)$ se le llama el **polinomio cromático** de G .

Un par de propiedades útiles

Si $G_1, \dots, G_\kappa$ son las componentes conexas de G , entonces

$$\chi_G(k) = \prod_{i=1}^{\kappa} \chi_{G_i}(k).$$

Un par de propiedades útiles

Si $G_1, \dots, G_\kappa$ son las componentes conexas de G , entonces

$$\chi_G(k) = \prod_{i=1}^{\kappa} \chi_{G_i}(k).$$

Si G no tiene loops, entonces $\chi_G(k) = \chi_{G'}(k)$, donde G' es la *simplificación* de G , el grafo que se obtiene al suprimir todas las aristas paralelas de G .

La evaluación en $k = -1$

Como $\chi_G(\cdot)$ coincide con un polinomio, podemos abstraernos de la definición de coloraciones y evaluarlo en cualquier número.

¿Qué pasa si evaluamos $\chi_G(k)$ en $k = -1$?

La evaluación en $k = -1$

Como $\chi_G(\cdot)$ coincide con un polinomio, podemos abstraernos de la definición de coloraciones y evaluarlo en cualquier número.

¿Qué pasa si evaluamos $\chi_G(k)$ en $k = -1$?

Ejercicio

Para todo grafo G se cumple que

$$(-1)^{n(G)} \chi_G(-1) \geq 0.$$

La evaluación en $k = -1$

Como $\chi_G(\cdot)$ coincide con un polinomio, podemos abstraernos de la definición de coloraciones y evaluarlo en cualquier número.

¿Qué pasa si evaluamos $\chi_G(k)$ en $k = -1$?

Ejercicio

Para todo grafo G se cumple que

$$(-1)^{n(G)} \chi_G(-1) \geq 0.$$

¿Tiene sentido decir que esa función es un problema de conteo?

La evaluación en $k = -1$

Como $\chi_G(\cdot)$ coincide con un polinomio, podemos abstraernos de la definición de coloraciones y evaluarlo en cualquier número.

¿Qué pasa si evaluamos $\chi_G(k)$ en $k = -1$?

Ejercicio

Para todo grafo G se cumple que

$$(-1)^{n(G)} \chi_G(-1) \geq 0.$$

¿Tiene sentido decir que esa función es un problema de conteo?

La función $G \mapsto (-1)^{n(G)} \chi_G(-1)$ pertenece a $\#P$.

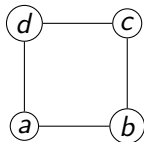
¿**Por qué?** ¿Cómo podríamos demostrar algo como esto?

Orientaciones acíclicas

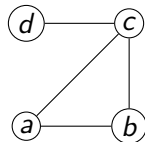
Una **orientación acíclica** de un grafo G es una asignación de una dirección a cada arista de modo que no se forme ningún ciclo dirigido. Al número de orientaciones acíclicas de G lo denotaremos $\#OA(G)$.

Orientaciones acíclicas

Una **orientación acíclica** de un grafo G es una asignación de una dirección a cada arista de modo que no se forme ningún ciclo dirigido. Al número de orientaciones acíclicas de G lo denotaremos $\#OA(G)$.

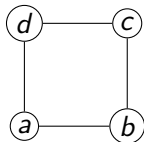


$$\#OA(G) = 2^4 - 2 = 14$$

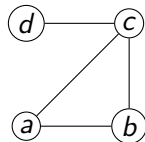


Orientaciones acíclicas

Una **orientación acíclica** de un grafo G es una asignación de una dirección a cada arista de modo que no se forme ningún ciclo dirigido. Al número de orientaciones acíclicas de G lo denotaremos $\#OA(G)$.



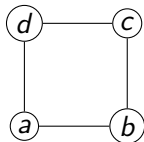
$$\#OA(G) = 2^4 - 2 = 14$$



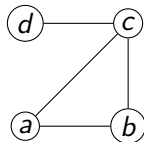
$$\#OA(G) = 2^4 - 2 \cdot 2 = 12$$

Orientaciones acíclicas

Una **orientación acíclica** de un grafo G es una asignación de una dirección a cada arista de modo que no se forme ningún ciclo dirigido. Al número de orientaciones acíclicas de G lo denotaremos $\#OA(G)$.



$$\#OA(G) = 2^4 - 2 = 14$$



$$\#OA(G) = 2^4 - 2 \cdot 2 = 12$$

Ejercicio

$\#OA(G) > 0$ si y solo si G no tiene loops.

Supresión-contracción para orientaciones acíclicas

Recursión de supresión-contracción para orientaciones acíclicas

$$\#OA(G) = \begin{cases} 1 & \text{si } m(G) = 0 \\ 0 & \text{si } G \text{ tiene loops} \\ \#OA(G \setminus e) + \#OA(G/e) & \text{si } e \in E \text{ no es loop} \end{cases} .$$

Supresión-contracción para orientaciones acíclicas

Recursión de supresión-contracción para orientaciones acíclicas

$$\#OA(G) = \begin{cases} 1 & \text{si } m(G) = 0 \\ 0 & \text{si } G \text{ tiene loops} \\ \#OA(G \setminus e) + \#OA(G/e) & \text{si } e \in E \text{ no es loop} \end{cases}.$$

Demostración (idea).

Digamos que e conecta los vértices u y v . Definimos dos conjuntos

- A_1 : OAs de $G \setminus e$ para las que existe un camino dirigido de u a v o de v a u (disyunción exclusiva).
- A_2 : OAs de $G \setminus e$ para las que u y v son mutuamente inalcanzables.

Supresión-contracción para orientaciones acíclicas

Recursión de supresión-contracción para orientaciones acíclicas

$$\#OA(G) = \begin{cases} 1 & \text{si } m(G) = 0 \\ 0 & \text{si } G \text{ tiene loops} \\ \#OA(G \setminus e) + \#OA(G/e) & \text{si } e \in E \text{ no es loop} \end{cases}.$$

Demostración (idea).

Digamos que e conecta los vértices u y v . Definimos dos conjuntos

- A_1 : OAs de $G \setminus e$ para las que existe un camino dirigido de u a v o de v a u (disyunción exclusiva).
- A_2 : OAs de $G \setminus e$ para las que u y v son mutuamente inalcanzables.

Se cumple que $\#OA(G) = |A_1| + 2|A_2| = \#OA(G \setminus e) + |A_2|$.

Supresión-contracción para orientaciones acíclicas

Recursión de supresión-contracción para orientaciones acíclicas

$$\#OA(G) = \begin{cases} 1 & \text{si } m(G) = 0 \\ 0 & \text{si } G \text{ tiene loops} \\ \#OA(G \setminus e) + \#OA(G/e) & \text{si } e \in E \text{ no es loop} \end{cases}.$$

Demostración (idea).

Digamos que e conecta los vértices u y v . Definimos dos conjuntos

- A_1 : OAs de $G \setminus e$ para las que existe un camino dirigido de u a v o de v a u (disyunción exclusiva).
- A_2 : OAs de $G \setminus e$ para las que u y v son mutuamente inalcanzables.

Se cumple que $\#OA(G) = |A_1| + 2|A_2| = \#OA(G \setminus e) + |A_2|$.

Por último, veremos que hay una biyección entre A_2 y las OAs de G/e . $\square$

La biyección entre A_2 y las OAs de G/e

Sea w el vértice que resulta de contraer la arista e en G .

Dada una orientación $\sigma \in A_2$, definimos $f(\sigma)$ como la orientación de G/e que se obtiene agregando la arista e y luego realizando la contracción.

La biyección entre A_2 y las OAs de G/e

Sea w el vértice que resulta de contraer la arista e en G .

Dada una orientación $\sigma \in A_2$, definimos $f(\sigma)$ como la orientación de G/e que se obtiene agregando la arista e y luego realizando la contracción.

Veamos que $f(\sigma)$ es acíclica. Si tiene un ciclo, entonces este contiene a w (pues σ era una OA de $G \setminus e$). Si $w, v_1, \dots, v_\ell$ es un ciclo de $f(\sigma)$, como σ es acíclica, necesariamente $v_1, \dots, v_\ell$ es un camino dirigido (en σ) de u a v o de v a u . Pero u y v son mutuamente inalcanzables en σ .

La biyección entre A_2 y las OAs de G/e

Sea w el vértice que resulta de contraer la arista e en G .

Dada una orientación $\sigma \in A_2$, definimos $f(\sigma)$ como la orientación de G/e que se obtiene agregando la arista e y luego realizando la contracción.

Veamos que $f(\sigma)$ es acíclica. Si tiene un ciclo, entonces este contiene a w (pues σ era una OA de $G \setminus e$). Si $w, v_1, \dots, v_\ell$ es un ciclo de $f(\sigma)$, como σ es acíclica, necesariamente $v_1, \dots, v_\ell$ es un camino dirigido (en σ) de u a v o de v a u . Pero u y v son mutuamente inalcanzables en σ .

f es una función inyectiva. ¿Y la sobreyectividad?

La biyección entre A_2 y las OAs de G/e

Sea w el vértice que resulta de contraer la arista e en G .

Dada una orientación $\sigma \in A_2$, definimos $f(\sigma)$ como la orientación de G/e que se obtiene agregando la arista e y luego realizando la contracción.

Veamos que $f(\sigma)$ es acíclica. Si tiene un ciclo, entonces este contiene a w (pues σ era una OA de $G \setminus e$). Si $w, v_1, \dots, v_\ell$ es un ciclo de $f(\sigma)$, como σ es acíclica, necesariamente $v_1, \dots, v_\ell$ es un camino dirigido (en σ) de u a v o de v a u . Pero u y v son mutuamente inalcanzables en σ .

f es una función inyectiva. ¿Y la sobreyectividad?

Sea τ una orientación acíclica de G/e . Construimos una orientación σ de $G \setminus e$ así: a cada arista entre $x \in V$ y u o v la orientamos según cómo están orientadas las aristas entre x y w en τ , y para las demás aristas copiamos la orientación. En σ se cumple que u y v son mutuamente inalcanzables, pues sino τ no sería acíclica. Entonces $\sigma \in A_2$ y $f(\sigma) = \tau$.

El puente entre $\#OA$ y el polinomio cromático

Teorema (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#OA(G)$$

El puente entre $\#OA$ y el polinomio cromático

Teorema (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#OA(G)$$

Inducción en $m(G)$. Si $m(G) = 0$, entonces $\chi_G(k) = k^{n(G)}$ y

$$(-1)^{n(G)} \chi_G(-1) = (-1)^{2n(G)} = 1 = \#OA(G).$$

El puente entre $\#OA$ y el polinomio cromático

Teorema (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#OA(G)$$

Inducción en $m(G)$. Si $m(G) = 0$, entonces $\chi_G(k) = k^{n(G)}$ y

$$(-1)^{n(G)} \chi_G(-1) = (-1)^{2n(G)} = 1 = \#OA(G).$$

Ahora supongamos que $m(G) \geq 1$. Si G tiene loops, entonces ambos lados son iguales a 0.

El puente entre $\#OA$ y el polinomio cromático

Teorema (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#OA(G)$$

Inducción en $m(G)$. Si $m(G) = 0$, entonces $\chi_G(k) = k^{n(G)}$ y

$$(-1)^{n(G)} \chi_G(-1) = (-1)^{2n(G)} = 1 = \#OA(G).$$

Ahora supongamos que $m(G) \geq 1$. Si G tiene loops, entonces ambos lados son iguales a 0. En caso contrario, sea $e \in E$ no loop. Entonces

$$\begin{aligned} (-1)^{n(G)} \chi_G(-1) &= (-1)^{n(G)} (\chi_{G \setminus e}(-1) - \chi_{G/e}(-1)) \\ &= (-1)^{n(G)} \chi_{G \setminus e}(-1) + (-1)^{n(G)-1} \chi_{G/e}(-1) \\ &= \#OA(G \setminus e) + \#OA(G/e) = \#OA(G) \end{aligned}$$

$\#P$ y las interpretaciones combinatoriales

Corolario (¡ahora sí!)

La función $G \mapsto (-1)^{n(G)} \chi_G(-1)$ pertenece a $\#P$.

$\#P$ y las interpretaciones combinatoriales

Corolario (¡ahora sí!)

La función $G \mapsto (-1)^{n(G)} \chi_G(-1)$ pertenece a $\#P$.

Podemos calcular la función $G \mapsto (-1)^{n(G)} \chi_G(-1)$ **contando objetos**.
El valor de dicha función tiene una **interpretación combinatorial**.

#P y las interpretaciones combinatoriales

Corolario (¡ahora sí!)

La función $G \mapsto (-1)^{n(G)} \chi_G(-1)$ pertenece a #P.

Podemos calcular la función $G \mapsto (-1)^{n(G)} \chi_G(-1)$ **contando objetos**.
El valor de dicha función tiene una **interpretación combinatorial**.

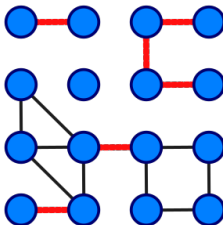
¿Qué acabamos de hacer?

- Consideramos dos problemas de naturaleza distinta que cumplen recursiones de supresión-contracción.
- Agregamos factores para entrelazar las recursiones.
- Probamos una igualdad que conecta ambas funciones.

Obtenemos un puente que permite traspasar resultados sobre la complejidad de ambos problemas.

Loops, coloops y aristas comunes

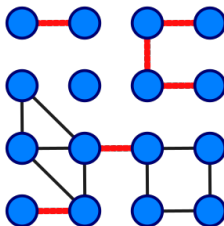
$e \in E$ es un **coloop** (o *punte*, o *istmo*) de G si $\kappa(G \setminus e) > \kappa(G)$.



User: Booyabazooka, Wikimedia Commons, CC BY-SA 3.0

Loops, coloos y aristas comunes

$e \in E$ es un **coloop** (o *punte*, o *istmo*) de G si $\kappa(G \setminus e) > \kappa(G)$.



User: Booyabazooka, Wikimedia Commons, CC BY-SA 3.0

Diremos que $e \in E$ es **común** si no es ni loop ni coloop.

$c(G)$ denotará el número de coloos de G , y $\ell(G)$, el número de loops. Claramente G tiene aristas comunes si y solo si $c(G) + \ell(G) < m(G)$.

El polinomio de fiabilidad

Sea $0 < p < 1$ un real. Definimos $R_G(p)$ como la probabilidad de que el número de componentes conexas de G no aumente si cada arista se suprime con probabilidad p y se mantiene con probabilidad $1 - p$ (como eventos independientes).

El polinomio de fiabilidad

Sea $0 < p < 1$ un real. Definimos $R_G(p)$ como la probabilidad de que el número de componentes conexas de G no aumente si cada arista se suprime con probabilidad p y se mantiene con probabilidad $1 - p$ (como eventos independientes).

Recursión de supresión-contracción para la función de fiabilidad

$$R_G(p) = \begin{cases} p \cdot R_{G \setminus e}(p) + (1 - p) \cdot R_{G/e}(p) & \text{si } e \text{ es arista común} \\ (1 - p)^{c(G)} & \text{si } c(G) + \ell(G) = m(G) \end{cases}.$$

El polinomio de fiabilidad

Sea $0 < p < 1$ un real. Definimos $R_G(p)$ como la probabilidad de que el número de componentes conexas de G no aumente si cada arista se suprime con probabilidad p y se mantiene con probabilidad $1 - p$ (como eventos independientes).

Recursión de supresión-contracción para la función de fiabilidad

$$R_G(p) = \begin{cases} p \cdot R_{G \setminus e}(p) + (1 - p) \cdot R_{G/e}(p) & \text{si } e \text{ es arista común} \\ (1 - p)^{c(G)} & \text{si } c(G) + \ell(G) = m(G) \end{cases}.$$

Corolario

$R_G(p)$ es un polinomio en p con coeficientes enteros.

A $R_G(p)$ se le conoce como el **polinomio de fiabilidad** de G .

Definición expandida del polinomio de Tutte

El **polinomio de Tutte** de G es el siguiente polinomio en $\mathbb{Z}[x, y]$:

$$T_G(x, y) = \sum_{A \subseteq E} (x - 1)^{\kappa_G(A) - \kappa_G(E)} (y - 1)^{\kappa_G(A) + |A| - n(G)},$$

donde $\kappa_G(A)$ denota el número de componentes conexas del grafo (V, A) .

Definición expandida del polinomio de Tutte

El **polinomio de Tutte** de G es el siguiente polinomio en $\mathbb{Z}[x, y]$:

$$T_G(x, y) = \sum_{A \subseteq E} (x-1)^{\kappa_G(A) - \kappa_G(E)} (y-1)^{\kappa_G(A) + |A| - n(G)},$$

donde $\kappa_G(A)$ denota el número de componentes conexas del grafo (V, A) .

El polinomio de Tutte del grafo loop $\mathcal{L}$

$$T_{\mathcal{L}}(x, y) = \underbrace{(x-1)^{1-1} (y-1)^{1+0-1}}_{A=\emptyset} + \underbrace{(x-1)^{1-1} (y-1)^{1+1-1}}_{A=E(\mathcal{L})} = y.$$

Definición expandida del polinomio de Tutte

El **polinomio de Tutte** de G es el siguiente polinomio en $\mathbb{Z}[x, y]$:

$$T_G(x, y) = \sum_{A \subseteq E} (x-1)^{\kappa_G(A) - \kappa_G(E)} (y-1)^{\kappa_G(A) + |A| - n(G)},$$

donde $\kappa_G(A)$ denota el número de componentes conexas del grafo (V, A) .

El polinomio de Tutte del grafo loop $\mathcal{L}$

$$T_{\mathcal{L}}(x, y) = \underbrace{(x-1)^{1-1} (y-1)^{1+0-1}}_{A=\emptyset} + \underbrace{(x-1)^{1-1} (y-1)^{1+1-1}}_{A=E(\mathcal{L})} = y.$$

El polinomio de Tutte del grafo coloop $\mathcal{I}$

$$T_{\mathcal{I}}(x, y) = \underbrace{(x-1)^{2-1} (y-1)^{2+0-2}}_{A=\emptyset} + \underbrace{(x-1)^{1-1} (y-1)^{1+1-2}}_{A=E(\mathcal{I})} = x.$$

La recursión fundamental del polinomio de Tutte

Factorización de loops

Si $e \in E$ es un loop, entonces

$$T_G(x, y) = y \cdot T_{G \setminus e}(x, y).$$

Factorización de coloops

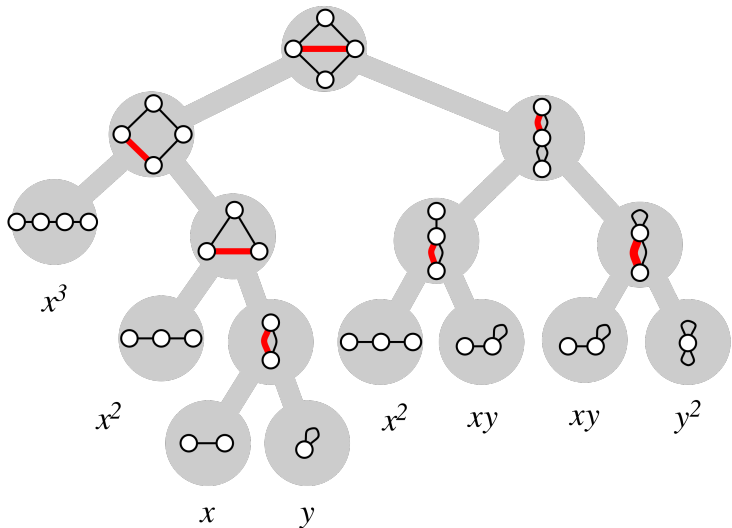
Si $e \in E$ es un coloop, entonces

$$T_G(x, y) = x \cdot T_{G/e}(x, y).$$

Supresión-contracción

$$T_G(x, y) = \begin{cases} T_{G \setminus e}(x, y) + T_{G/e}(x, y) & \text{si } e \text{ es arista común} \\ x^{c(G)} y^{\ell(G)} & \text{si } c(G) + \ell(G) = m(G) \end{cases}.$$

Ejemplo de la recursión del polinomio de Tutte



User: Thore Husfeldt, Wikimedia Commons, CC BY-SA 3.0

Un par de propiedades interesantes

Factorización

Si $G = G_1 \cup G_2$, donde G_1 y G_2 comparten a lo sumo un vértice, entonces

$$T_G(x, y) = T_{G_1}(x, y) \cdot T_{G_2}(x, y).$$

Un par de propiedades interesantes

Factorización

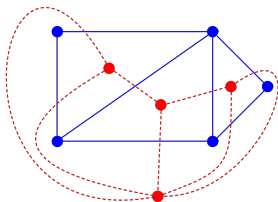
Si $G = G_1 \cup G_2$, donde G_1 y G_2 comparten a lo sumo un vértice, entonces

$$T_G(x, y) = T_{G_1}(x, y) \cdot T_{G_2}(x, y).$$

Dualización de grafos planares

Si G es planar y conexo, y G^* es su grafo dual, entonces

$$T_{G^*}(x, y) = T_G(y, x).$$



User: Self, Wikimedia Commons, CC0

Todo está en el polinomio de Tutte

Sea $\mathcal{G}$ el conjunto de clases de isomorfismo de grafos.

Propiedad universal del polinomio de Tutte

Sea $F: \mathcal{G} \rightarrow \mathbb{C}[\alpha, \beta, \gamma, x, y]$ una función que satisface lo siguiente:

- i. $F(G) = \gamma^{n(G)}$ si G no tiene aristas.
- ii. $F(G) = x \cdot F(G/e)$ si e es un coloop.
- iii. $F(G) = y \cdot F(G \setminus e)$ si e es un loop.
- iv. $F(G) = \alpha \cdot F(G/e) + \beta \cdot F(G \setminus e)$ si e es una arista común.

Si α y β no son nulos, entonces se cumple la siguiente igualdad:

$$F(G) = \gamma^{\kappa(G)} \alpha^{n(G) - \kappa(G)} \beta^{\kappa(G) + m(G) - n(G)} \cdot T_G \left(\frac{x}{\alpha}, \frac{y}{\beta} \right).$$

Algunas evaluaciones del polinomio de Tutte

El polinomio cromático

$$\chi_G(k) = (-1)^{n(G)-\kappa(G)} k^{\kappa(G)} \cdot T_G(1-k, 0).$$

Algunas evaluaciones del polinomio de Tutte

El polinomio cromático

$$\chi_G(k) = (-1)^{n(G)-\kappa(G)} k^{\kappa(G)} \cdot T_G(1-k, 0).$$

En particular, tomando $k = 3$, el número de 3-coloraciones de un grafo conexo G es $|3 \cdot T_G(-2, 0)|$.

El polinomio de fiabilidad

$$R_G(p) = (1-p)^{n(G)-\kappa(G)} p^{\kappa(G)+m(G)-n(G)} \cdot T_G\left(1, \frac{1}{p}\right).$$

Algunas evaluaciones del polinomio de Tutte

El polinomio cromático

$$\chi_G(k) = (-1)^{n(G)-\kappa(G)} k^{\kappa(G)} \cdot T_G(1-k, 0).$$

En particular, tomando $k = 3$, el número de 3-coloraciones de un grafo conexo G es $|3 \cdot T_G(-2, 0)|$.

El polinomio de fiabilidad

$$R_G(p) = (1-p)^{n(G)-\kappa(G)} p^{\kappa(G)+m(G)-n(G)} \cdot T_G\left(1, \frac{1}{p}\right).$$

Ejemplos de evaluaciones puntuales

- $T_G(2, 1)$ es el número de subconjuntos acíclicos de aristas de G .
- Si G es conexo, $T_G(1, 1)$ es el número de árboles generadores de G .
- $T_G(0, 2)$ es el número de orientaciones totalmente cíclicas de G .

La complejidad de calcular un coeficiente específico

El siguiente problema es #P-completo:

Entrada: un grafo G y dos enteros no negativos i, j dados en binario.

Salida: El coeficiente del término $x^i y^j$ en $T_G(x, y)$.

La complejidad de calcular un coeficiente específico

El siguiente problema es #P-completo:

Entrada: un grafo G y dos enteros no negativos i, j dados en binario.

Salida: El coeficiente del término $x^i y^j$ en $T_G(x, y)$.

Demostración.

Es claro que la función tiene salidas no negativas viendo la recursión de supresión-contracción de $T_G(x, y)$. Si $i + j > m(G)$, el coeficiente es 0. En caso contrario, tenemos que $i, j \in O(m(G))$.

La complejidad de calcular un coeficiente específico

El siguiente problema es $\#P$ -completo:

Entrada: un grafo G y dos enteros no negativos i, j dados en binario.

Salida: El coeficiente del término $x^i y^j$ en $T_G(x, y)$.

Demostración.

Es claro que la función tiene salidas no negativas viendo la recursión de supresión-contracción de $T_G(x, y)$. Si $i + j > m(G)$, el coeficiente es 0.

En caso contrario, tenemos que $i, j \in O(m(G))$.

Para pertenencia a $\#P$, lo que hacemos es establecer un orden en las aristas y aplicar supresión-contracción siempre en la primera arista que sigue siendo común. Podemos usar el no determinismo para adivinar la rama de la recursión (eligiendo entre ramas de supresión o ramas de contracción) y luego verificar si terminamos en un grafo con i coloops, j loops y sin aristas comunes, en cuyo caso aceptamos.

La complejidad de calcular un coeficiente específico

El siguiente problema es $\#P$ -completo:

Entrada: un grafo G y dos enteros no negativos i, j dados en binario.

Salida: El coeficiente del término $x^i y^j$ en $T_G(x, y)$.

Demostración.

Es claro que la función tiene salidas no negativas viendo la recursión de supresión-contracción de $T_G(x, y)$. Si $i + j > m(G)$, el coeficiente es 0.

En caso contrario, tenemos que $i, j \in O(m(G))$.

Para pertenencia a $\#P$, lo que hacemos es establecer un orden en las aristas y aplicar supresión-contracción siempre en la primera arista que sigue siendo común. Podemos usar el no determinismo para adivinar la rama de la recursión (eligiendo entre ramas de supresión o ramas de contracción) y luego verificar si terminamos en un grafo con i coloops, j loops y sin aristas comunes, en cuyo caso aceptamos.

A continuación veremos que $G \mapsto T_G(x, y)$ es $\#P$ -hard, de lo que se sigue que este problema también es $\#P$ -hard. □

#P-equivalencia del polinomio de Tutte

La función $G \mapsto T_G(x, y)$ es #P-equivalente.

#P-equivalencia del polinomio de Tutte

La función $G \mapsto T_G(x, y)$ es #P-equivalente.

Demostración.

Para #P-hardness, basta recordar que podemos recuperar el número de 3-coloraciones de G a partir de $T_G(x, y)$.

#P-equivalencia del polinomio de Tutte

La función $G \mapsto T_G(x, y)$ es #P-equivalente.

Demostración.

Para #P-hardness, basta recordar que podemos recuperar el número de 3-coloraciones de G a partir de $T_G(x, y)$.

Para #P-easiness, podemos hacer $O(m(G)^2)$ llamadas al problema de calcular coeficientes de $T_G(x, y)$, y con ello recuperamos todo el polinomio. □

El teorema de dicotomía del polinomio de Tutte

Teorema (Jaeger, Vertigan y Welsh, 1990)

Sea $x_0, y_0 \in \mathbb{Q}$. La función $G \mapsto T_G(x_0, y_0)$ es computable en tiempo polinomial si (x_0, y_0) está en el conjunto

$$\left\{ (x, y) \in \mathbb{Q}^2 \mid xy = x + y \right\} \cup \left\{ (1, 1), (-1, -1), (0, -1), (-1, 0) \right\},$$

y es #P-hard en otro caso.

El teorema de dicotomía del polinomio de Tutte

Teorema (Jaeger, Vertigan y Welsh, 1990)

Sea $x_0, y_0 \in \mathbb{Q}$. La función $G \mapsto T_G(x_0, y_0)$ es computable en tiempo polinomial si (x_0, y_0) está en el conjunto

$$\left\{ (x, y) \in \mathbb{Q}^2 \mid xy = x + y \right\} \cup \left\{ (1, 1), (-1, -1), (0, -1), (-1, 0) \right\},$$

y es #P-hard en otro caso.

Teorema (Vertigan y Welsh, 1992)

Fuera de los casos fáciles del teorema anterior, si además G es un grafo bipartito planar, entonces la función $G \mapsto T_G(x_0, y_0)$ es computable en tiempo polinomial si y solo si $x_0 y_0 = x_0 + y_0 + 1$, y, si no, sigue siendo #P-hard.

Referencias I



R. P. Stanley,

Acyclic orientations of graphs,

Discrete Mathematics, 5(2):171–178, 1973.



F. Jaeger, D. L. Vertigan, and D. J. A. Welsh,

On the computational complexity of the Jones and Tutte polynomials,

Mathematical Proceedings of the Cambridge Philosophical Society, 108(1):35–53, 1990.



D. L. Vertigan and D. J. A. Welsh,

The computational complexity of the Tutte plane: the bipartite case,

Combinatorics, Probability and Computing, 1(2):181–187, 1992.

Referencias II



J. A. Ellis-Monaghan and C. Merino,
Graph Polynomials and Their Applications I: The Tutte Polynomial,
in *Structural Analysis of Complex Networks*, Birkhäuser Boston, pp. 219–255, 2011.



A. J. Goodall,
The Tutte polynomial and related polynomials,
Lecture notes, Charles University, 2013/2014.